

Seq2Seq Model — Encoder-Decoder

Sequence-to-Sequence (Seq2Seq) Model — Encoder-Decoder

◆ Sequence-to-Sequence (Seq2Seq) কী?

Sequence-to-Sequence (Seq2Seq) হলো এমন একটি নিউরাল নেটওয়ার্ক আর্কিটেকচার যা একটি **input sequence** (শব্দের সিরিজ) নিয়ে একটি **output sequence** তৈরি করে।

👉 Input ও Output দুটোই sequence (ক্রমযুক্ত ডেটা)

🎯 কোথায় ব্যবহার হয়?

- 🌐 Machine Translation (বাংলা → ইংরেজি)
- 📄 Text Summarization
- 💬 Chatbot
- 🎤 Speech Recognition
- ❓ Question Answering

🏗️ মূল গঠন: **Encoder–Decoder Architecture**

Seq2Seq মডেল সাধারণত দুইটি অংশ নিয়ে গঠিত:

- 1 **Encoder**
- 2 **Decoder**

Input Sentence → Encoder → Context Vector → Decoder → Output Sentence

🟦 ১. **Encoder** কী করে?

কাজ:

ইনপুট বাক্য পড়ে সেটাকে একটি **context vector**-এ রূপান্তর করে।

কীভাবে কাজ করে?

- ইনপুটের প্রতিটি শব্দ একে একে পড়ে (সাধারণত RNN/LSTM/GRU দিয়ে)
- প্রতিটি ধাপে একটি **hidden state** তৈরি হয়

- শেষ hidden state-টাই হয় **context vector**

উদাহরণ

Input: "I eat rice"

Step 1: I → h1

Step 2: eat → h2

Step 3: rice → h3

Final hidden state (h3) = Context Vector

👉 এই Context Vector-এ পুরো বাক্যের অর্থ compressed অবস্থায় থাকে।

২. Decoder কী করে?

কাজ:

Context Vector ব্যবহার করে আউটপুট বাক্য তৈরি করে — এক শব্দ করে।

কীভাবে কাজ করে?

- প্রথমে **<start>** token ইনপুট হিসেবে নেয়
- Context Vector দিয়ে প্রথম শব্দ তৈরি করে
- তারপর তৈরি করা শব্দ আবার ইনপুট হিসেবে ব্যবহার করে
- এভাবে চলতে থাকে **<end>** token আসা পর্যন্ত

উদাহরণ (**Translation**)

Input: "I eat rice"

Encoder → Context Vector

Decoder:

<start> → "আমি"

<start> আমি → "ভাত"

<start> আমি ভাত → "খাই"

<start> আমি ভাত খাই → **<end>**

Output: "আমি ভাত খাই"

Context Vector কী?

- এটি একটি fixed-size vector
- পুরো ইনপুট বাক্যের অর্থ একটিমাত্র ভেক্টরে ধরে রাখে
- এটি Encoder-এর শেষ hidden state

! Seq2Seq Model-এর সমস্যা

১ Information Bottleneck

- পুরো বাক্যের অর্থ একটি fixed-size vector-এ compress করা হয়
- বড় বাক্যের ক্ষেত্রে তথ্য হারিয়ে যায়

২ Long Sentence Problem

দীর্ঘ বাক্যে শুরু দিকের তথ্য শেষ পর্যন্ত ঠিকমতো থাকে না

৩ Vanishing Gradient

RNN ব্যবহার করলে gradient ধীরে ধীরে ছোট হয়ে যায়

↔ Attention যোগ করলে কী হয়?

এই সমস্যাগুলো সমাধান করতে **Attention Mechanism** যোগ করা হয়।

👉 তখন Decoder শুধু Context Vector নয়, ইনপুটের প্রতিটি hidden state-এর দিকে আলাদাভাবে মনোযোগ দেয়।

vs RNN vs Seq2Seq

Feature	Basic RNN	Seq2Seq
Output	Single output	Sequence output
Structure	Single network	Encoder + Decoder
Translation	না	হ্যাঁ

Encoder–Decoder with Attention



Encoder–Decoder with Attention — Full Concept

1. Basic Encoder–Decoder Recap (Without Attention)

◆ Encoder

- ইনপুট sequence একে একে পড়ে (RNN/LSTM/GRU)
- প্রতিটি ধাপে hidden state তৈরি করে
- শেষ hidden state = **Context Vector (c)**

◆ Decoder

- Context Vector ব্যবহার করে আউটপুট sequence তৈরি করে
- এক শব্দ করে generate করে



সমস্যা

- পুরো ইনপুটের অর্থ একটি fixed-size vector-এ compress হয়
- বড় বাক্যে তথ্য হারিয়ে যায় (Information Bottleneck)

2. Attention কেন দরকার?

ধরো বাক্য:

"আমি ঢাকায় জন্মেছিলাম এবং সেখানে বড় হয়েছি তাই আমি ভালো বাংলা বলতে পারি"

শেষের শব্দ "বাংলা" বুঝতে শুরু দিকের তথ্য দরকার।

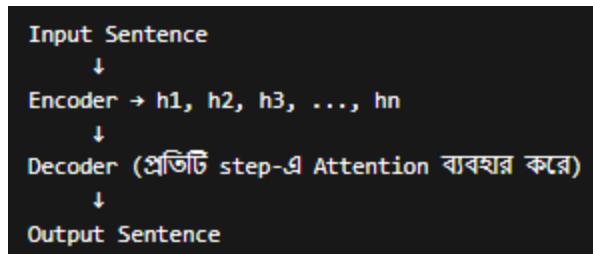


Basic Seq2Seq-এ Decoder শুধু Context Vector পায়



Attention-এ Decoder ইনপুটের প্রতিটি শব্দের দিকে আলাদা মনোযোগ দিতে পারে

3. Encoder–Decoder with Attention Architecture



এখানে গুরুত্বপূর্ণ পার্থক্য:

- ✗ আগে শুধু শেষ hidden state ব্যবহার হত
- ✓ এখন Encoder-এর সব **hidden state** ব্যবহার হয়

4.Encoder কী করে?

ইনপুট:"I eat rice"

Encoder তৈরি করে:

- $h_1 \rightarrow I$
- $h_2 \rightarrow eat$
- $h_3 \rightarrow rice$

এখন আমাদের কাছে আছে: $H = \{h_1, h_2, h_3\}$

5.Decoder কীভাবে Attention ব্যবহার করে?

ধরো Decoder "ভাত" শব্দ তৈরি করছে।

Step 1: Similarity Score হিসাব

Decoder-এর বর্তমান hidden state (s_t) এবং প্রতিটি encoder hidden state (h_i)-এর মধ্যে similarity হিসাব করা হয়।

score(s_t, h_i)

এটি হতে পারে:

- Dot Product
- Additive (Bahdanau)

- Scaled Dot Product

Step 2: Attention Weights বের করা

```
a_ti = softmax(score(s_t, h_i))
```

এটি probability-এর মতো কাজ করে।

উদাহরণ:

Word	Attention Weight
I	0.1
eat	0.7
rice	0.2

Step 3: Context Vector তৈরি

```
c_t = \sum (a_{ti} \times h_i)
```

এটি হলো weighted sum।

Step 4: Output তৈরি

```
y_t = Decoder(s_t, c_t)
```

অর্থাৎ:

Decoder Hidden State + Context Vector $\rightarrow$ Output word

6.Mathematical Formulation

Attention Score:

`</> Code`

```
e_ti = score(s_t-1, h_i)
```

Attention Weight:

`</> Code`

```
a_ti = exp(e_ti) / Σ exp(e_tj)
```

Context Vector:

`</> Code`

```
c_t = Σ a_ti h_i
```

Decoder Output:

`</> Code`

```
s_t = f(s_t-1, y_t-1, c_t)
```

7. Visualization Idea

ধরো:

`</> Code`

Input:	I	eat	rice
	↓	↓	↓
	h1	h2	h3

Decoder যখন "খাই" তৈরি করছে:

`</> Code`

Weights: 0.1 0.2 0.7

Decoder যখন "ভাত" তৈরি করছে:

`</> Code`

Weights: 0.1 0.8 0.1

প্রতিটি আউটপুট শব্দের জন্য আলাদা attention map তৈরি হয়।

8.Types of Attention

1 Bahdanau Attention (Additive Attention)

- RNN-based Seq2Seq-এ ব্যবহৃত
- Neural network দিয়ে similarity হিসাব করে

2 Luong Attention (Multiplicative)

- Dot product ব্যবহার করে
- দ্রুত

3 Self-Attention

- Transformer-এ ব্যবহৃত
- একই sequence-এর ভেতরে attention

9.Training vs Inference

Training

- Teacher Forcing ব্যবহার হয়
- সঠিক আগের শব্দ decoder-এ দেওয়া হয়

Inference

- নিজের তৈরি করা শব্দ পরের input হয়
- Autoregressive generation

10.Advantages of Attention

- ✓ Long sentence handle করতে পারে
- ✓ Information bottleneck দূর করে
- ✓ Interpretability দেয় (Attention Map দেখা যায়)
- ✓ Translation quality improve করে

Limitations

- ✗ এখনো RNN sequential
- ✗ Parallel processing করা যায় না
- ✗ খুব বড় sequence-এ slow

👉 এই সমস্যার সমাধান: **Transformer**

Without vs With Attention

Feature	Without Attention	With Attention
Context	Single Vector	Dynamic Vector (প্রতিটি step-এ আলাদা)
Long Sentence	দুর্বল	শক্তিশালী
Translation Quality	কম	বেশি

Transformer Full note

Transformer Full note

১. ANN, CNN, RNN — রিক্যাপ

ANN (Artificial Neural Network)

- সবচেয়ে basic neural network
- ইনপুট নেয় → হিডেন লেয়ারে প্রসেস করে → আউটপুট দেয়
- প্রতিটি নিউরন আগের লেয়ারের সব নিউরনের সাথে connected (Fully Connected)
- সমস্যা: Sequential data (যেমন বাক্য, সময় সিরিজ) ভালো handle করতে পারে না কারণ এটি ক্রম বোঝে না

CNN (Convolutional Neural Network)

- মূলত Image/Visual data-র জন্য
- Filter দিয়ে ছবির feature বের করে (যেমন edge, shape, texture)
- Spatial pattern বোঝে কিন্তু Sequential/Text data-র জন্য উপযুক্ত না

RNN (Recurrent Neural Network)

- Sequential data-র জন্য তৈরি
- প্রতিটি step-এ আগের step-এর **hidden state** মনে রাখে
- কাজ করে এভাবে: প্রতিটি শব্দ একে একে নেয়, আগের শব্দের মেমোরি সাথে রাখে
- সমস্যা ১ — **Vanishing Gradient**: বড় বাক্যে দূরের শব্দের তথ্য হারিয়ে যায়
- সমস্যা ২ — **Sequential Processing**: একসাথে প্রসেস করতে পারে না, একটার পর একটা করতে হয়, তাই ধীরগতি
- সমস্যা ৩: "আমি ঢাকায় জন্মেছিলাম এবং সেখানে বড় হয়েছি, তাই আমি ভালো বাংলা বলতে পারি" — এখানে "বাংলা" বুঝতে অনেক আগের context দরকার, RNN সেটা ধরে রাখতে পারে না

LSTM (Long Short-Term Memory)

- RNN-এর উন্নত version
- Cell State এবং Gate ব্যবহার করে গুরুত্বপূর্ণ তথ্য বেশি মনে রাখে
- তবুও খুব বড় sequence-এ সমস্যা থাকে এবং parallel processing করা যায় না

২. Encoder-Decoder Architecture

মূল ধারণা

- **Encoder**: ইনপুট বাক্য পড়ে একটি **Context Vector** তৈরি করে

- **Context Vector:** পুরো ইনপুটের অর্থ একটি fixed-size vector-এ ধরে রাখে
- **Decoder:** Context Vector থেকে আউটপুট বাক্য তৈরি করে, একটা একটা শব্দ করে

উদাহরণ (Translation)

```
ইনপুট: "I eat rice"
  ↓ Encoder
Context Vector [0.5, 0.3, 0.8, ...]
  ↓ Decoder
আউটপুট: "আমি ভাত খাই"
```

সমস্যা

- Context Vector-এর size নির্দিষ্ট (fixed)
- বড় বাক্য হলে সব তথ্য একটি ছোট vector-এ ধরা যায় না
- Information Bottleneck তৈরি হয়
- দীর্ঘ বাক্যে শুরুর শব্দের তথ্য শেষে হারিয়ে যায়

৩. Attention Mechanism

কেন দরকার হলো?

- Fixed context vector-এর সীমাবদ্ধতা দূর করতে
- Decoder যখন প্রতিটি শব্দ তৈরি করে, তখন সে ইনপুটের সব শব্দের দিকে আলাদাভাবে মনোযোগ দিতে পারবে

কীভাবে কাজ করে?

Decoder যখন একটি আউটপুট শব্দ তৈরি করবে, তখন:

১. ইনপুটের প্রতিটি শব্দের সাথে Decoder-এর বর্তমান অবস্থার **similarity** হিসাব করে
২. যে শব্দের সাথে বেশি মিল, তাকে বেশি ওজন (**Attention Score**) দেয়
৩. সব শব্দের weighted sum নিয়ে একটি **context vector** তৈরি করে
৪. এই context vector দিয়ে আউটপুট শব্দ তৈরি করে

উদাহরণ

```
ইনপুট:  "I    eat    rice"
          ↓      ↓      ↓
Weights: 0.1  0.2  0.7  ← "খাই" বলার সময়
          0.1  0.8  0.1  ← "ভাত" বলার সময়
          0.8  0.1  0.1  ← "আমি" বলার সময়
```

Attention Score হিসাব

Score = dot product(Decoder hidden state, Encoder hidden state)

Attention Weight = Softmax(Score)

Context = $\sum(\text{Attention Weight} \times \text{Encoder hidden state})$

সুবিধা

- প্রতিটি আউটপুট শব্দের জন্য আলাদা context তৈরি হয়
- দীর্ঘ বাক্যেও কাজ করে
- মডেল কোন শব্দের দিকে মনোযোগ দিচ্ছে সেটা visualize করা যায় (interpretable)

Attention-এর সমস্যা (RNN + Attention)

- এখনো Sequential processing (RNN ব্যবহার করছে)
- Training এখনো ধীর
- এই সমস্যার সমাধান হলো **Transformer**

8. Transformer Architecture

মূল বিপ্লব

- **"Attention is All You Need"** — 2017 সালে Google-এর গবেষণাপত্র
- RNN, LSTM সব বাদ দিয়ে শুধু Attention দিয়ে কাজ করে
- সব শব্দ একসাথে (**Parallel**) প্রসেস করা যায়
- Training অনেক দ্রুত

Transformer-এর মূল উপাদান

১. Input Embedding
২. Positional Encoding
৩. Multi-Head Self-Attention
৪. Feed-Forward Network
৫. Layer Normalization
৬. Encoder Stack
৭. Decoder Stack
৮. Cross-Attention

৫. Self-Attention (নিজের সাথে নিজের Attention)

ধারণা

- Encoder-Decoder Attention-এ Decoder ইনপুটের দিকে মনোযোগ দিত
- **Self-Attention**-এ একটি বাক্যের প্রতিটি শব্দ, সেই একই বাক্যের অন্য সব শব্দের দিকে মনোযোগ দেয়

উদাহরণ

"The animal didn't cross the street because it was too tired"

- "it" মানে কি "animal" নাকি "street"?
- Self-Attention মডেলকে বুঝতে সাহায্য করে যে "it" → "animal" এর দিকে বেশি মনোযোগ দেওয়া উচিত

কীভাবে কাজ করে — Query, Key, Value

প্রতিটি শব্দের জন্য তিনটি vector তৈরি হয়:

Query (Q) — "আমি কী খুঁজছি?"

- এই শব্দটি বুঝতে অন্য কোন শব্দের তথ্য দরকার?

Key (K) — "আমার কাছে কী আছে?"

- প্রতিটি শব্দের একটি "পরিচয়পত্র" যেটা দিয়ে বোঝা যায় এটা কতটা relevant

Value (V) — "আমি আসলে কী তথ্য দেবো?"

- যদি এই শব্দ relevant হয়, তাহলে এই তথ্যটি পাঠাবে

Attention Score হিসাব (Step by Step)

Step 1: প্রতিটি শব্দের Embedding $\times$ Weight Matrix দিয়ে Q, K, V তৈরি করো

Word Embedding $\times$ W_q = Query

Word Embedding $\times$ W_k = Key

Word Embedding $\times$ W_v = Value

Step 2: Score হিসাব করো

Score = $Q \times K(\text{transpose})$

Step 3: Scale করো (স্থিতিশীলতার জন্য)

Scaled Score = $\text{Score} / \sqrt{(\text{dimension of } K)}$

Step 4: Softmax দিয়ে probability-তে রূপান্তর

Attention Weight = $\text{Softmax}(\text{Scaled Score})$

Step 5: Value-এর সাথে গুণ করো

Output = $\text{Attention Weight} \times V$

Matrix আকারে (একসাথে সব শব্দ)

$$\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k}) \times V$$

৬. Multi-Head Attention

কেন দরকার?

একটি Attention head শুধু একটি দিক দেখে। কিন্তু একটি বাক্য বোঝার জন্য অনেক দিক বোঝা দরকার, যেমন:

- কোন শব্দটি grammatically সম্পর্কিত?
- কোন শব্দের অর্থগত মিল আছে?
- কোন শব্দটি coreference (সে, এটা, তারা)?

কীভাবে কাজ করে?

১. একই ইনপুটের জন্য **h** টি আলাদা আলাদা **Attention** চালানো হয় (যেমন $h=8$)
২. প্রতিটি head আলাদা W_q, W_k, W_v matrix ব্যবহার করে
৩. প্রতিটি head আলাদা aspect শেখে
৪. সব head-এর output **Concatenate** করা হয়
৫. একটি final Linear layer দিয়ে combine করা হয়

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head1}, \text{head2}, \dots, \text{headh}) \times W_o$$

যেখানে $\text{head}_i = \text{Attention}(Q \times W_{qi}, K \times W_{ki}, V \times W_{vi})$

উদাহরণ

"আমি সে ব্যক্তিকে দেখেছিলাম যে আমার বই নিয়েছিল"

- Head 1 → "সে" কে refer করছে?
- Head 2 → "আমার" কার?
- Head 3 → verb-object relationship কী?
- Head 4 → কোন শব্দগুলো একসাথে clause তৈরি করছে?

৭. Masked Attention (Decoder-এ)

কেন দরকার?

- Decoder যখন training-এ আউটপুট তৈরি করে, তখন সে ভবিষ্যতের শব্দ দেখতে পারবে না
- কারণ real life-এ ভবিষ্যত শব্দ জানা থাকে না
- যদি দেখতে পায়, তাহলে সে শুধু copy করবে, কিছু শিখবে না

কীভাবে কাজ করে?

- Attention Score matrix-এ ভবিষ্যতের position-গুলোকে $-\infty$ (**negative infinity**) দিয়ে mask করা হয়
- Softmax-এর পরে এই position-গুলোর weight হয় 0
- ফলে মডেল শুধু আগের শব্দগুলো দেখতে পায়

"আমি ভাত খাই" তৈরি করার সময়:

- "আমি" তৈরি করতে: শুধু <start> token দেখতে পায়
- "ভাত" তৈরি করতে: শুধু "আমি" দেখতে পায়
- "খাই" তৈরি করতে: "আমি ভাত" দেখতে পায়

৮. Positional Encoding

সমস্যা

- Transformer সব শব্দ একসাথে নেয়, তাই শব্দের ক্রম হারিয়ে যায়
- "আমি তোমাকে ভালোবাসি" এবং "তোমাকে আমি ভালোবাসি" — দুটো কি একই?
- শুধু Embedding দিলে মডেল ক্রম বুঝতে পারবে না

সমাধান

প্রতিটি শব্দের Embedding-এর সাথে তার **Position** সম্পর্কিত একটি **Vector** যোগ করা হয়

Positional Encoding Formula

```
PE(pos, 2i) = sin(pos / 10000^(2i/d))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d))

যেখানে:
pos = শব্দের অবস্থান (0, 1, 2, ...)
i = dimension index
d = embedding dimension
```

কেন sin/cos?

- sin এবং cos periodic, তাই যেকোনো দূরত্বের মধ্যে relative position বোঝা যায়
- দুটি শব্দের মধ্যে পার্থক্য শেখা যায়
- যেকোনো দৈর্ঘ্যের sequence handle করা যায়

Final Input

Final Input = Word Embedding + Positional Encoding

৯. Encoder Architecture

উপাদান (প্রতিটি **Encoder Layer-এ**)

১. **Multi-Head Self-Attention** — প্রতিটি শব্দ অন্য সব শব্দের দিকে মনোযোগ দেয়
২. **Add & Norm (Residual Connection + Layer Normalization)** — gradient flow ভালো রাখে
৩. **Feed-Forward Network** — প্রতিটি শব্দ আলাদাভাবে process হয়
৪. **Add & Norm** — আবার

Residual Connection কী?

Output = LayerNorm(x + Sublayer(x))

- ইনপুটকে সরাসরি আউটপুটে যোগ করা হয়
- Vanishing gradient সমস্যা কমায়
- Deep network train করা সহজ হয়

Encoder Stack

- সাধারণত 6টি Encoder Layer একের পর এক থাকে (Original Transformer-এ)
- প্রতিটি layer আরও গভীর understanding তৈরি করে
- শেষ Encoder Layer-এর আউটপুট Decoder-এ যায়

Encoder কী করে?

ইনপুটের প্রতিটি শব্দকে **contextual representation** দেয়। একই শব্দ ভিন্ন context-এ ভিন্ন representation পাবে।

উদাহরণ: "bank" শব্দটি "river bank" এবং "bank account"-এ ভিন্ন representation পাবে।

১০. Feed-Forward Network (Transformer-এর ভেতরে)

কীভাবে কাজ করে?

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- দুটি Linear layer
- মাঝে ReLU activation
- প্রতিটি শব্দের জন্য আলাদাভাবে apply হয় (position-wise)

কেন দরকার?

- Self-Attention শব্দের মধ্যে **relationship** বোঝে
- FFN সেই relationship থেকে **feature/pattern** শেখে
- আরও complex transformation সম্ভব করে

Dimension

- ইনপুট: $d_{\text{model}} = 512$
- Hidden: $d_{\text{ff}} = 2048$ (4× বড়)
- আউটপুট: $d_{\text{model}} = 512$

১১. Decoder Architecture

উপাদান (প্রতিটি Decoder Layer-এ)

১. **Masked Multi-Head Self-Attention** — ভবিষ্যত দেখতে পারে না
২. **Add & Norm**
৩. **Cross-Attention** — Encoder-এর আউটপুটের দিকে মনোযোগ দেয়
৪. **Add & Norm**
৫. **Feed-Forward Network**
৬. **Add & Norm**

Cross-Attention কী?

- Decoder-এর Query আসে Decoder থেকে
- Key এবং Value আসে **Encoder**-এর আউটপুট থেকে
- এভাবে Decoder ইনপুট বাক্যের যেকোনো অংশের দিকে মনোযোগ দিতে পারে

Cross-Attention:

Q = Decoder hidden state

K = Encoder output

V = Encoder output

এটাই মূলত Translation-এ কাজ করে

উদাহরণ (Translation)

"আমি ভাত খাই" → "I eat rice"

- "I" তৈরি করতে: "আমি"-র দিকে বেশি মনোযোগ
- "eat" তৈরি করতে: "খাই"-র দিকে বেশি মনোযোগ
- "rice" তৈরি করতে: "ভাত"-র দিকে বেশি মনোযোগ

১২. Transformer Training

Teacher Forcing

Training-এ Decoder-কে সঠিক আগের শব্দ দেওয়া হয়, নিজের তৈরি করা শব্দ না।

Target: "I eat rice"

ধাপ ১: Input = <start>, Expected = "I"

ধাপ ২: Input = <start> I, Expected = "eat"

ধাপ ৩: Input = <start> I eat, Expected = "rice"

ধাপ ৪: Input = <start> I eat rice, Expected = <end>

কেন Teacher Forcing?

- শুরুতে মডেল ভুল শব্দ তৈরি করে
- ভুল শব্দের উপর নির্ভর করে পরের শব্দ শিখলে ভুল জমতে থাকে
- Teacher Forcing দিয়ে এই Error Propagation বন্ধ করা যায়
- Training দ্রুত ও stable হয়

Loss Function

Cross-Entropy Loss = $-\sum y_{\text{true}} \times \log(y_{\text{pred}})$

প্রতিটি position-এ সঠিক শব্দ কতটা probable তা দিয়ে loss হিসাব হয়।

১৩. Transformer Testing (Inference)

কীভাবে কাজ করে?

Training-এর মতো না। এখানে কোনো Teacher Forcing নেই।

ধাপ ১: Encoder পুরো ইনপুট একসাথে প্রসেস করে

ধাপ ২: Decoder শুরু করে <start> token দিয়ে

ধাপ ৩: প্রথম শব্দ তৈরি হয়

ধাপ ৪: সেই শব্দ আবার Decoder-এ ইনপুট হয়

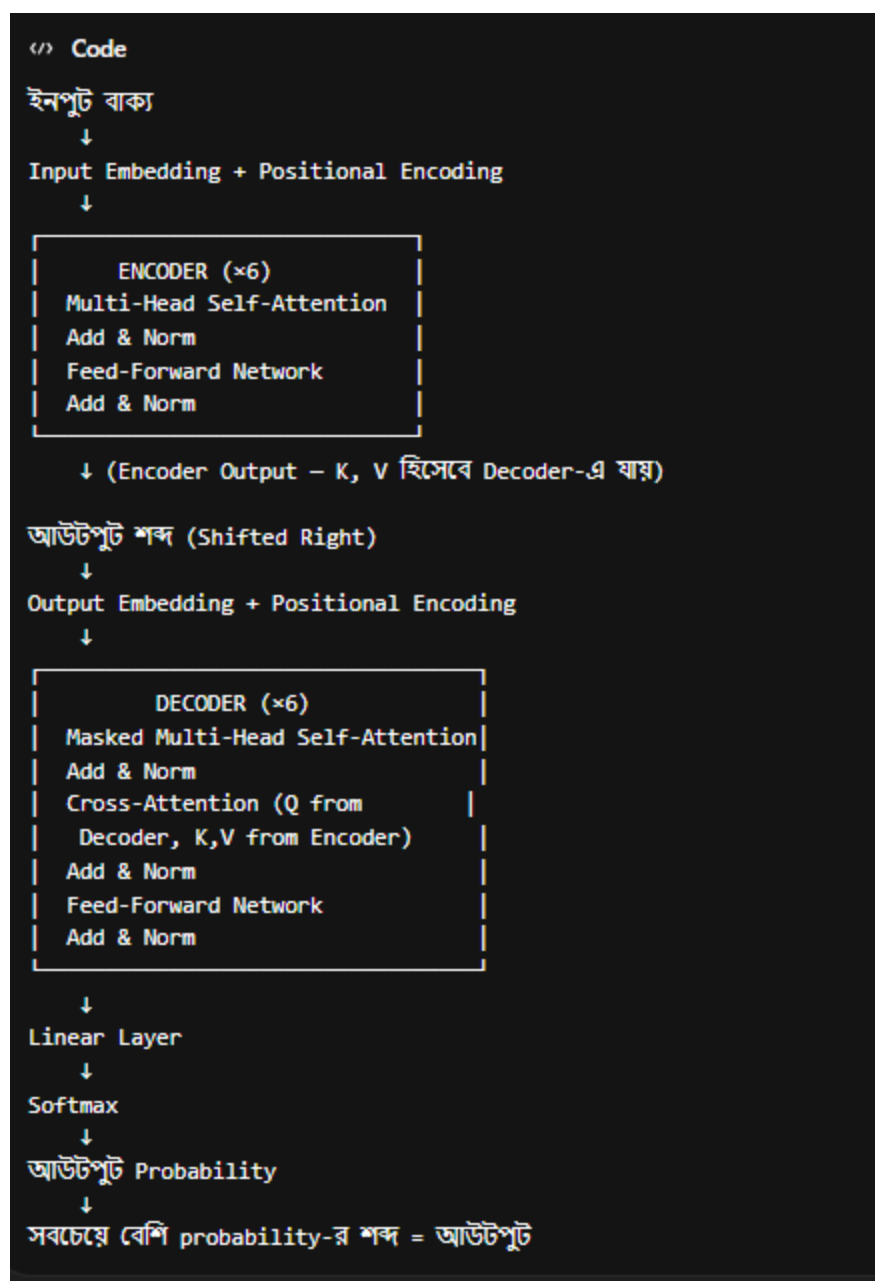
ধাপ ৫: পরের শব্দ তৈরি হয়

ধাপ ৬: <end> token আসা পর্যন্ত চলতে থাকে

Autoregressive Generation

- একটি শব্দ তৈরি হয়, সেটি আবার ইনপুট হয়, পরের শব্দ তৈরি হয়
- এভাবে একটার পর একটা শব্দ তৈরি হতে থাকে
- এটাই ChatGPT, Claude-সহ সব LLM-এর কাজ করার পদ্ধতি

১৪. পুরো Transformer — একনজরে



১৫. Transformer কেন এত গুরুত্বপূর্ণ?

বিষয়	RNN/LSTM	Transformer
Processing	Sequential (ধীর)	Parallel (দ্রুত)
Long Sequence	দুর্বল	শক্তিশালী

Training Speed	ধীর	দ্রুত
Scalability	সীমিত	অসীম (GPT-4 = 1.8 Trillion parameter)
Context	সীমিত	বিশাল (128K+ tokens)

Transformer দিয়ে তৈরি বিখ্যাত মডেল

- **BERT** — Encoder-only (Google, Text Classification, Q&A)
- **GPT series** — Decoder-only (OpenAI, Text Generation)
- **T5, BART** — Encoder-Decoder (Translation, Summarization)
- **Claude, Gemini** — Large Language Models

Video link: [Deep Learning Complete Course | Part 4 | Transformers & Attention Mechanism Completely Explained](#)
Blog Link: [The Illustrated Transformer – Jay Alammar – Visualizing machine learning one concept at a time.](#)